Cowboy Storage: CBFS and Runner Attached Storage

Technical Whitepaper

Charles DePue, Martin Aceto

September 25, 2026 — v0.10.4

Contents

1 Abstract **3**

2 Introduction **3**

3 Architectural Overview **4**
3.1 Terminology . . . 4
3.2 Design Rationale . . . 5
3.3 Differences vs. On-chain State . . . 5

4 Volumes and the Account Model **6**

5 The CBFS Data Plane **6**
5.1 Object Representation . . . 8
5.2 Encryption . . . 9
5.3 Manifest . . . 9
5.4 Placement . . . 10
5.5 Erasure Coding . . . 11
5.6 Transport . . . 11
5.7 FUSE Mount . . . 12

6 The CapToken Authorization Model **12**
6.1 Runner CapTokens (chain-issued) . . . 13
6.2 Owner CapTokens (client-signed) . . . 14
6.3 DelegationCert . . . 15
6.4 Direct-Client Grants . . . 15
6.5 Access Modes . . . 16
6.6 Scoping and Concurrency . . . 17

7 Volume Lifecycle **17**
7.1 Create (control plane) . . . 17
7.2 Attach (control plane) . . . 18
7.3 Write (object transfer) . . . 18
7.4 Read (object transfer) . . . 18
7.5 Commit and Publication . . . 19
7.6 Delete, Grace, Restore . . . 20

8 Relay Nodes **21**
8.1 Registration and Stake . . . 21

© Cowboy Labs, Inc. 2026

8.2 Repair . . . 22
8.2.1 Repair Local Shards . . . 22
8.2.2 Replace Unavailable Relays . . . 22
8.3 Drain and Peer-Push . . . 23

9 Proof of Retrievability **23**
9.1 Challenge Protocol . . . 23
9.2 Outcomes . . . 24

10 Billing, Rent, and Eviction **25**
10.1 Fee Components . . . 25
10.2 Relay Earnings . . . 26
10.3 Settlement . . . 27
10.4 Comparison with State Rent . . . 29

11 On-Chain Control Plane **29**
11.1 Storage Manager (`0x0A`) . . . 29
11.2 Relay Registry (`0x0B`) . . . 31
11.3 Cache Freshness . . . 32

12 State Transition Function (Storage Subset) **32**

13 Security Considerations **33**
13.1 Client-side encryption . . . 33
13.2 Shard ID opacity . . . 33
13.3 Durability and Scoped Writes . . . 33
13.4 Relay eclipse and withholding . . . 33
13.5 Sybil resistance . . . 34
13.6 Key compromise . . . 34
13.7 Key-delivery boundary . . . 34
13.8 Replay and nonce reuse . . . 34

14 Parameters (Genesis Defaults) **34**

15 Relationship to the Technical Whitepaper **37**

© Cowboy Labs, Inc. 2026

1 Abstract

This document specifies Cowboy's storage layer: the **Cowboy File System (CBFS)** and **Runner Attached Storage (RAS)**. CBFS is a decentralized, encrypted, erasure-coded storage substrate operated by a permissionless network of **Relay Nodes**. RAS is the Cowboy protocol layer that mounts CBFS under account-scoped **Volumes**, anchors volume state on-chain via compact **StorageCommitments**, and mediates access to volumes through cryptographic **capability tokens** (CapTokens).

CBFS transfers object data off-chain without a transaction per object access. RAS anchors volume ownership, manifest publication, billing, challenges, and eviction on-chain. Private volumes encrypt their contents before distribution to Relay Nodes. A **delegation keypair** model separates cold wallet authority from hot storage authority, allowing owner-direct data operations to be authorized locally and verified offline by Relay Nodes against cached chain state.

This specification covers the volume lifecycle, CapToken and delegation cryptography, Reed-Solomon erasure coding, Relay Node registration and repair, Proof-of-Retrievability challenges, per-epoch storage rent, and storage parameters. It is a complement to the **Cowboy Technical Whitepaper**, which is the normative reference for consensus, execution, economics, and actor semantics. The Relationship to the Technical Whitepaper section defines the governing references.

2 Introduction

Actors on Cowboy are autonomous Python programs with on-chain storage subject to state rent and a 1 MiB default quota (extendable to 8 MiB via a storage bond) — see the Technical Whitepaper §3.2, §4.4. That model is sized for state, not for data. Modern autonomous agents need to persist working memory, cache retrieval traces, hold model adapters, stage artifacts between jobs, and serve large public assets. None of those fit naturally into on-chain state: they are too large, too churn-heavy, or too privacy-sensitive to push into a consensus-critical key/value store.

Cowboy separates consensus state from bulk storage:

- **State** — small, consensus-critical key/value data that participates directly in the state transition function. Billed per cell, subject to rent, stored in QMDB, and bounded by per-actor quotas.
- **Storage** — large, potentially private data, durably kept across a permissionless Relay Node network and referenced on-chain only by compact commitments. Billed per byte per epoch in a separate market, mediated by capability tokens rather than transactions.

© Cowboy Labs, Inc. 2026

CBFS is the storage substrate. It is designed to be usable standalone (as a plain distributed filesystem) and as the data plane for Cowboy, with the same binaries and wire protocol in both configurations. RAS is the control plane: it defines the rules under which Cowboy accounts own volumes, grant runners access, settle storage rent, and register and slash Relay Nodes.

3 Architectural Overview

This overview introduces the components. Requirements and their rationale appear together in the sections that follow.

This document uses **MUST/SHOULD/MAY** as defined in RFC 2119. Parameters marked **governance-tunable** can be changed via the on-chain governance mechanism specified in Technical Whitepaper §11. In all cases, the Technical Whitepaper is the normative reference for consensus, execution, and economic parameters outside the storage layer.

The detailed storage contract is CIP-9; storage pricing and settlement follow CIP-31. The authorization rules below summarize CIP-9, including its direct-client grant contract.

3.1 Terminology

- **Volume:** A named, account-scoped storage namespace. Each volume is a private (encrypted) or public (plaintext) container of objects identified by path keys.
- **Object:** A single blob within a volume, identified by its path. Encrypted client-side for private volumes; plaintext for public ones.
- **Relay Node:** A network participant that stores erasure-coded shards of volume data, answers reads, participates in repair, and serves Proof-of-Retrievability challenges. A distinct role from Validators and Runners.
- **Shard:** A fragment of an erasure-coded object. An object is split into K data + M parity shards; any K reconstruct the ciphertext.
- **Manifest:** The authoritative, Merkle-committed index of a volume's contents. Encrypted for private volumes, plaintext for public ones, stored as content-addressed DAG nodes on Relay Nodes.
- **StorageCommitment:** The on-chain record of a volume's existence, owner, manifest root, erasure parameters, size, billing state, and status.
- **PlacementRecord:** The off-chain, mutable record binding a shard ID to its assigned Relay Nodes. Replicated across assigned nodes; versioned with compare-and-swap semantics.
- **CapToken:** A bearer credential authorizing volume operations. Runner tokens are issued by the chain at job dispatch; owner-auth tokens are signed by a delegated client key and

© Cowboy Labs, Inc. 2026

admitted through volume ownership or an explicit direct-client grant. Scope and validation depend on the authorization path.

- **DelegationCert:** A wallet-signed certificate that binds an Ed25519 CBFS key to a Cowboy account, scoped to storage use only, with a short (30-day) expiry.
- **Storage Manager (`0x0A`):** The Cowboy system actor that maintains StorageCommitments, mints runner CapTokens, handles per-epoch billing, and governs volume status transitions.
- **Relay Registry (`0x0B`):** The Cowboy system actor that maintains Relay Node profiles, staking, health, and repair/challenge coordination.

3.2 Design Rationale

Actor workloads need mutable working state, private data, shared artifacts, and filesystem access across short-lived runners. CBFS addresses these requirements through:

- **Off-chain object transfers:** clients exchange shards directly with Relay Nodes; publication and lifecycle changes use the chain.
- **Scoped authorization:** job attachments bind storage access to a runner; owners and explicitly authorized clients use locally signed tokens.
- **Committed manifests:** an on-chain root identifies the authoritative volume contents served by relays.
- **Repair without decryption:** erasure shards and placement metadata let relays repair private data without holding the DEK.
- **Filesystem access:** FUSE lets runner workloads use ordinary file operations against persistent volumes.

The following sections specify these mechanisms and their limits.

3.3 Differences vs. On-chain State

Aspect	On-chain State	CBFS / RAS
Typical size	Bytes to kilobytes	Kilobytes to hundreds of GiB
Privacy	Public to all validators	Encrypted per volume (private mode)
Fees	Cycles, Accesses, Cells, and state rent	Storage rent and transfer fees; chain mutations also pay execution fees

© Cowboy Labs, Inc. 2026

Aspect	On-chain State	CBFS / RAS
Durability	Consensus replication across all validators	Reed-Solomon K+M across Relay Nodes
Access control	Actor and account authorization	Runner tokens, ownership, and direct-client grants
Data path	State mutations execute on-chain; read-only RPCs query committed state	Object transfers use relay RPCs; manifest publication uses a transaction
Latency	Mutations wait for consensus; read-only queries use RPC	Transfers depend on relay round trips; publication waits for consensus

4 Volumes and the Account Model

Each volume is identified by:

```
volume_id = keccak256(owner_address || utf8_bytes(volume_name))
```

`volume_name` MUST be UTF-8, NFC-normalized, 1–64 bytes long, and match `^[a-zA-Z0-9_\-.]+$`. It MUST NOT begin or end with `.` or `-`. Each volume has a fixed **visibility** set at creation and never changed:

- **PRIVATE:** Objects are AES-256-GCM encrypted under a per-volume DEK. The manifest is encrypted. Reads and writes require CapTokens.
- **PUBLIC:** Objects are plaintext. The manifest is plaintext. Reads are open to any party without a CapToken; writes still require one.

An account MUST NOT exceed `MAX_VOLUMES_PER_ACCOUNT`. Each volume MUST NOT exceed `MAX_OBJECTS_PER_VOLUME = 1,000,000` objects or `MAX_VOLUME_SIZE = 100 GiB`; each object MUST NOT exceed `MAX_OBJECT_SIZE = 1 GiB`. Object paths are UTF-8 strings of up to `MAX_OBJECT_PATH_LENGTH = 512 bytes`. The full addressing scheme for an object is:

```
/{owner_address}/{volume_name}/{object_path}
```

5 The CBFS Data Plane

CBFS is the substrate that stores and serves volume data. Its implementation is divided into the following crates:

Crate	Responsibility
`cbfs-types`	Shared protocol types: `VolumeId`, `ShardId`, `NodeId`, `PlacementRecord`, `ObjectDescriptor`, `RpcRequest/Response`, `Operation`.
`cbfs-crypto`	AES-256-GCM, BLAKE3, DEK generation and envelope wrapping.
`cbfs-erasure`	Reed-Solomon over GF(2^8) via `reed-solomon-erasure` v6 with SIMD.
`cbfs-transport`	QUIC client/server (`quinn`) with TLS 1.3 and bincode-framed RPCs.
`cbfs-store`	Sled-backed blob store on Relay Nodes.
`cbfs-manifest`	Sorted client-side volume index and canonical content-chunked Merkle DAG.
`cbfs-placement`	Persistent `PlacementStore` for shard-to-node assignments with CAS and replica merge.
`cbfs-hooks`	Pluggable traits (`AuthProvider`, `ManifestRegistry`, `NodeSelector`, `MeteringSink`) with standalone and Cowboy-mode implementations.
`cbfs-auth`	Cowboy-facing auth runtime: delegation loading, request signing, owner token minting, relay and chain-state caches.
`cbfs-sdk`	High-level Volume API: create, open, put, get, commit, with local manifest and placement synchronization.
`cbfs-fuse`	POSIX FUSE mount (inode table, write-back cache, sync daemon, extended attributes).
`cbfs-node`	The Relay Node daemon: QUIC server, garbage collection, heartbeats, repair, drain.
`cbfs-cli`	Operator CLI: `login`, `auth`, `volume`, `put`, `get`, `mount`, `commit`, `rebalance`.

© Cowboy Labs, Inc. 2026

5.1 Object Representation

Each object is described by an **ObjectDescriptor** stored in the manifest:

Field	Type	Meaning
`object_path`	`String`	≤ 512 bytes, UTF-8
`write_id`	`[u8; 16]`	Fresh CSPRNG per write, isolates versions
`content_hash`	`[u8; 32]`	BLAKE3 of plaintext
`ciphertext_hash`	`[u8; 32]`	BLAKE3 of ciphertext before erasure coding
`encryption_nonce`	`[u8; 12]`	Fresh random per write (AES-GCM invariant)
`size_bytes`	`u64`	Plaintext size
`ciphertext_size`	`u64`	Pre-padding ciphertext size (required for erasure truncation)
`shard_id`	`[u8; 32]`	`BLAKE3(volume_id || object_path || write_id)` — opaque to Relay Nodes
`erasure_k`, `erasure_m`	`u8`, `u8`	Per-object erasure parameters
`shard_hashes`	`[[u8; 32]; K+M]`	BLAKE3 of each shard, used for integrity and PoR

The opacity of `shard_id` — Relay Nodes see a hash, not a path — preserves privacy: they cannot learn which object a shard belongs to, nor detect overwrites. The Scoping and Concurrency section explains the resulting authorization limits.

Each write MUST use a fresh CSPRNG-generated `write_id` and record `content_hash = BLAKE3(plaintext)`. Private objects MUST also record `ciphertext_hash = BLAKE3(ciphertext)` and the size and padding metadata needed for reconstruction without decryption.

© Cowboy Labs, Inc. 2026

5.2 Encryption

A private volume's DEK MUST be 32 bytes drawn from a CSPRNG. Each private object MUST be encrypted with AES-256-GCM using a fresh random 12-byte nonce; deterministic nonces MUST NOT be used, and a (DEK, nonce) pair MUST NOT be reused. The AAD MUST bind the ciphertext to its descriptor's `volume_id`, `object_path`, and `write_id`; the authentication tag MUST be appended to the ciphertext per RFC 5116. Readers MUST verify `ciphertext_hash` before decryption and `content_hash` afterward. Public volumes store plaintext objects and manifests.

5.3 Manifest

The client indexes file, symlink, and directory entries by path. The committed representation MUST follow the canonical, content-chunked Merkle DAG in CIP-9 §3.1 (`cbfs/manifest/src/dag.rs`): entries are ordered by UTF-8 path bytes, grouped into leaf nodes, and reduced through interior nodes to one root. Each node's locator is the BLAKE3 hash of its canonical stored encoding; `StorageCommitment.manifest_root` is the top node's locator.

Private-volume nodes are individually encrypted before storage, so the root commits to ciphertext. Public-volume nodes are plaintext. Unchanged nodes retain their locators across commits; clients publish only newly added nodes, addressed by `manifest_node_shard_id(volume_id, locator)`. Readers MUST verify nodes against their expected locators. The canonical encoding, chunk boundaries, and encryption domains are specified in CIP-9 §3.1. See Commit and Publication for ordering and authority.

Each manifest DAG node MUST be stored as K+M erasure-coded shards at the volume-bound address:

```
manifest_node_shard_id(volume_id, locator) =
    BLAKE3("cbfs.manifest-node.v2" || volume_id || locator)
```

The hash input MUST be the domain's literal UTF-8 bytes followed by the 32-byte volume ID and the 32-byte node locator, without delimiters or a terminating NUL. Readers MUST start from the authoritative `StorageCommitment.manifest_root`, verify fetched nodes against their expected locators, and traverse the child locators. Private-volume nodes MUST be decrypted with the DEK during traversal. There is no well-known single manifest blob address.

Public-volume readers MAY use `GET_MANIFEST`, supplying the authoritative root and erasure parameters. Its response is an entry map, not the stored DAG encoding; readers MUST rebuild

© Cowboy Labs, Inc. 2026

the canonical DAG and verify its root against the authoritative root supplied in the request. This alternative does not apply to private volumes.

5.4 Placement

Shards are assigned to K+M distinct Relay Nodes via a **PlacementRecord**:

Field	Type	Meaning
`volume_id`	`[u8; 32]`	Volume binding checked against locally stored shard metadata
`shard_id`	`[u8; 32]`	Opaque shard identifier
`version`	`u64`	CAS version for atomic updates during repair
`assignments`	`[{shard_index: u8, node_id: [u8;32]}]`	Node per shard index
`erasure_k`, `erasure_m`	`u8`, `u8`	Duplicated from descriptor for repair without manifest
`ciphertext_size`	`u64`	Duplicated to enable reconstruction without decryption
`shard_hashes`	`[[u8; 32]; K+M]`	Duplicated for integrity checks during repair

PlacementRecords MUST remain mutable, versioned records separate from the manifest. They MUST duplicate the descriptor's erasure parameters, ciphertext size, and shard hashes so repair does not require decryption. They MUST be replicated across all assigned Relay Nodes; updates MUST use compare-and-swap through these RPCs:

- `PutPlacement(shard_id, record, expected_version)` — CAS store/update.
- `GetPlacement(shard_id)` — fetch.
- `ReplicatePlacement(shard_id, record)` — peer-to-peer replication, signed by the sending relay's identity key.

A PlacementRecord MUST carry its `volume_id`. When accepting an update or replication for a shard assigned to itself, a Relay Node MUST reject a record whose volume conflicts with that

shard's existing local volume binding. Placement updates cannot rebind stored bytes to another volume.

This externalization of placement — as opposed to embedding assignments in the (encrypted) manifest — is what allows repair workers to operate on private volumes without decryption rights.

Node selection is pluggable via the `NodeSelector` trait. The selector scores eligible relays by available capacity, geographic and operator diversity, health, reputation, and observed latency, then chooses K+M distinct destinations.

5.5 Erasure Coding

Cowboy uses Reed-Solomon over GF(2^8) via the `reed-solomon-erasure` v6 crate with the `simd-accel` feature. Defaults are K=4 data shards and M=2 parity shards, for 1.5× storage overhead.

For the default 4/6 scheme, an object is split into 6 shards distributed across 6 Relay Nodes; any 4 reconstruct the ciphertext, so the system tolerates any 2 simultaneous Relay Node failures per object. Accounts billed for `effective_size = raw_size × (K+M)/K`.

Implementations MUST support $2 \leq$ `K` $\leq$ `MAX_ERASURE_K = 16` and $1 \leq$ `M` $\leq$ `MAX_ERASURE_M = 8`. Accounts MAY choose higher redundancy at creation time.

Ciphertext MUST be padded to a multiple of K before splitting; readers MUST truncate reconstruction to the recorded `ciphertext_size`. Each shard MUST be hashed with BLAKE3, with the hash stored in both the ObjectDescriptor and PlacementRecord. Readers MUST verify shard hashes before reconstruction; repair workers MUST verify reconstructed shards before persistence.

5.6 Transport

Relay Nodes MUST use QUIC over TLS 1.3 on a single stream per RPC. Each frame is a `[version: u8][length: u32 big-endian][bincode payload]` tuple capped at `MAX_SHARD_TRANSFER_SIZE + 64 KiB` (default ~1 MiB). The RPC protocol is:

```
{ request_id: [u8;16], op: Operation, shard_id: [u8;32], shard_index: u8,
auth_token: Vec<u8>, payload: Vec<u8> }
```

Implementations MUST support the following `Operation` values: `PutShard`, `GetShard`, `DeleteShard`, `GetPlacement`, `PutPlacement`, `ReplicatePlacement`, `Ping`, `RelayHandshake`, `SyncPeers`, `PeerRelayPushShard`, `PeerPlacementUpdate`. Responses carry a matching

`request_id` and a status code (`Ok`, `ErrAuth`, `ErrNotFound`, `ErrCapacity`, `ErrInvalid`, `ErrConflict`).

Client-to-relay operations requiring authorization MUST validate a CapToken; public-volume reads remain open. Peer-relay authentication follows Drain and Peer-Push.

`RelayHandshake` MUST sign `keccak256("cbfs:relay-handshake:" || node_id || client_nonce || timestamp_ms_be)` with the relay's Ed25519 identity key. Clients MUST verify against the cached on-chain `relay_identity_pubkey` and reject timestamps outside ±30 seconds.

5.7 FUSE Mount

CBFS ships a FUSE filesystem (`cbfs-fuse`) that exposes a volume as a POSIX directory with write-back caching. Writes land instantly in an in-memory dirty cache; a background sync daemon encrypts, shards, and pushes to Relay Nodes every `DEFAULT_SYNC_INTERVAL = 5 seconds` (≥ `MIN_SYNC_INTERVAL = 1 second`). The local cache is bounded by `MAX_LOCAL_CACHE_SIZE = 10 GiB` per mount.

Extended attributes under the `cbfs.*` prefix expose shard and encryption metadata read-only:

Attribute	Value
`cbfs.status`	`committed` / `dirty` / `syncing`
`cbfs.encrypted`	`aes-256-gcm`
`cbfs.erasure`	`K+M` (e.g., `4+2`)
`cbfs.shards`	`shard_index@node_addr,...`
`cbfs.content_hash`	First 8 bytes of BLAKE3, hex

Cowboy-mode mounts refresh owner tokens before expiry while preserving the requested access mode, allowing long-lived sessions without weakening token scope.

6 The CapToken Authorization Model

Storage authorization combines chain-issued runner tokens with client-signed owner-auth tokens. CapTokens MUST be encoded as `[version: u8][bincode payload]`, with `0` for runner tokens and `1` for owner-auth tokens. Signing inputs that cover a token MUST include its version byte.

Relays validate credentials against authoritative, freshness-bounded chain state as described under Cache Freshness.

6.1 Runner CapTokens (chain-issued)

A **Runner CapToken** MUST be issued through the Storage Manager (`0x0A`) at job dispatch and stored on-chain. It MUST include the following fields:

Field	Type	Meaning
`version`	`u8`	`0`
`token_id`	`[u8; 16]`	Deterministically derived
`token_secret`	`bytes`	Entropy; validation requires secret match
`volume_id`	`[u8; 32]`	Target volume
`volume_name`	`String`	For logging, not validation
`runner_address`	`Address`	Restricts to a single runner
`access_mode`	`AccessMode`	`ReadOnly`, `WriteOnly`, `ReadWrite`
`path_prefix`	`String`	Canonical attachment prefix (e.g., `workspace/`); `""` means whole volume
`max_bytes`	`u64`	Per-token quota
`valid_from`, `valid_until`	`u64`, `u64`	Block heights
`mount`, `sync_interval`	`bool`, `u32`	Mount parameters

The `token_id` and `token_secret` are derived deterministically from job metadata, runner address, block height, attachment index, and VRF seed, preventing pre-image attacks. At runtime, a Relay Node validates a presented runner CapToken by looking up the stored copy at `StorageManager[token_id]` and checking byte-for-byte match plus TTL, runner address, volume status, and access mode. Prefix restrictions are enforced at attachment admission and checked by clients/coordinators; opaque shard RPCs do not expose paths for relay-side containment checks.

Runner CapTokens MUST have a validity window of at least `MIN_VOLUME_CAP_TOKEN_BLOCKS = 600,000` blocks (~7 days at 1 s blocks), sufficient for typical job durations.

6.2 Owner CapTokens (client-signed)

An **Owner CapToken** (`OwnerCapTokenV1`) is signed locally by a delegated CBFS key. The signer MUST be admitted either as the volume owner or through the Direct-Client Grants contract; a delegation cert alone conveys no cross-owner volume authority.

Field	Type	Meaning
`version`	`u8`	`1`
`volume_id`	`[u8; 32]`	Target volume
`volume_name`	`String`	For logging, not validation
`owner_address`	`Address`	Volume owner; MUST match the authoritative commitment owner
`access_mode`	`AccessMode`	`ReadOnly`, `WriteOnly`, `ReadWrite`
`path_prefix`	`String`	Requested scope; MUST be empty for a grant-scoped token
`valid_from_ms`, `valid_until_ms`	`u64`, `u64`	Unix milliseconds (wall-clock, short TTL)
`mount`, `sync_interval`	`bool`, `u32`	Mount parameters
`delegation_cert`	`DelegationCert`	Embedded wallet delegation cert
`owner_signature`	`bytes`	Ed25519 over all other fields, including the version byte

An owner-auth token MUST have a maximum TTL of **15 minutes** for reads and puts (`OWNER_TOKEN_OPEN_TTL_SECS = 900`), and **24 hours** for mounts (`OWNER_TOKEN_MOUNT_TTL_SECS = 86,400`). Long-lived operations refresh the token before expiry while preserving its scope. Relays validate owner-auth tokens without storing each token on-chain. They MUST check the wallet's secp256k1 signature on the embedded DelegationCert, the token's Ed25519 signature under the cert's key, cert expiry, `chain_id`, `network`, `aud == "cowboy:cbfs"`, and `scope == "cbfs"`. The cert MUST be registered and unrevoked for its wallet. Validation MUST also check the authoritative volume identity, owner and status, token TTL (±30 seconds clock

skew), and the requested operation against the access mode. Ownership admission requires `delegation_cert.wallet_address == commitment.owner`; otherwise the direct-client grant rules apply. Path scope MUST NOT be treated as a relay-enforced confidentiality boundary for opaque shards.

The token signature MUST cover the canonical `OwnerCapTokenSigningPayload`, including the version byte and embedded cert. RAS wire types and signing-byte helpers are owned by `cowboy-protocol-codec`; clients and validators MUST use that canonical encoding, as specified in CIP-9 §12.2.1.

6.3 DelegationCert

A **DelegationCert** binds an Ed25519 CBFS key to a Cowboy wallet. It MUST be signed with the wallet's secp256k1 key over the Keccak-256 digest of the canonical `DelegationCertSigningPayload`, and MUST expire no more than 30 days after issuance. It binds the wallet, delegated key, scope, service class, chain ID, network, audience, and expiry. The signing client registers the delegation on-chain; the cert can then accompany many owner-auth tokens.

The signed service class limits mount authority: a non-mount service delegation MUST NOT authorize a mount token. Relays MUST verify that the registered delegation matches the cert, including its service class and scope. Canonical fields and signing bytes are defined by `cowboy-protocol-codec` and consumed by the CBFS authorization layer.

The delegated CBFS key is stored encrypted at `~/.cbfs/cbfs_key.enc` under AES-256-GCM with a user-supplied passphrase (or environment variable for headless use), decrypted on demand into locked memory, and zeroed after signing. `cbfs auth rotate` replaces the key; `cbfs auth revoke` posts revocation to the Storage Manager. Cache Freshness bounds propagation to relays.

6.4 Direct-Client Grants

CIP-9 §7.7 and §7.7.1 define a single on-chain mount allowlist for cross-owner access. A grant without `grantee_signing_key` authorizes job dispatch only. A direct-client grant additionally binds the grantee's delegated Ed25519 signing key; its canonical path prefix MUST be empty, because opaque shard addressing and one DEK per private volume cannot enforce a narrower direct-client scope.

A grantee signs an `OwnerCapTokenV1` with its own DelegationCert and key, while `owner_address` names the volume's owner. In addition to ordinary token validation, a relay MUST require:

- The grant's grantee address matches the cert wallet by bare address, and `grantee_signing_key` matches the cert's delegated key.
- The token's mode is a subset of the grant's allowed operations, and its path prefix is empty.
- The grant is unexpired at the relay's block-height view. An expiring grant MUST be refused if that view is unavailable.
- `tee_required` is false; a direct client does not satisfy a runner TEE requirement.

A read grant permits volume-open metadata, committed-manifest discovery, shard and placement reads, and read-only proofs only when the request's volume is resolved from authoritative stored state. Unattributed shards and unscoped proofs MUST be refused. A write grant permits shard and placement uploads; a read-write grant permits both. Grants MUST NOT authorize replication, staging, finalization, manifest publication, deletion, or administrative operations. Volume-owner actions, including deposits, transfer, and allowlist changes, still require owner authorization. Uploaded data becomes visible only when the owner publishes a manifest referencing it.

For a private volume, direct access requires the grantee's separate X25519 encryption key and an HPKE-wrapped DEK bound to that key and the current `dek_version`, following CIP-9 §9.2.1. The encryption key and package MUST be supplied together, and the package MUST satisfy CIP-9's exact size and binding checks; public-volume grants MUST omit both. A private-volume write grantee holding the whole DEK has decryption capability: `WriteOnly` limits authorized operations, not what that key can decrypt.

A grantee opens by explicit owner and volume name. The response MUST supply the grantee's DEK package, never the owner's wrap; owner-wide enumeration still lists only owned volumes. Removing the grant, revoking the delegation, expiry, or ownership transfer ends access within the cache freshness bound. Transfer MUST clear the allowlist. Revocation does not erase plaintext or DEKs already obtained, and issued tokens remain bearer credentials until invalidated or expired.

6.5 Access Modes

Mode	Read	Write	List	Delete
`ReadOnly`	Yes	No	Yes	No
`WriteOnly`	No	Yes	No	No
`ReadWrite`	Yes	Yes	Yes	No

`Delete` is never granted to runners; it is an owner-only control-plane operation.

6.6 Scoping and Concurrency

Runner CapTokens are scoped by **volume**, **path prefix**, **byte quota**, and **time window**. CIP-9 §7.3.1 defines three enforcement boundaries:

1. **Issuance:** the Dispatcher MUST reject a write prefix that overlaps an already-valid write token on the same volume. Read-only tokens may overlap. Prefixes use the canonical attachment form: `""` for the whole volume, otherwise no leading slash and one trailing slash.
2. **Commit authority:** every commit MUST name the `prev_root` it extends; the chain rejects a mismatch with the current root. CIP-9 requires public-volume write-only commits to prove prefix confinement against the plaintext manifest delta. For private volumes, a write-only sub-agent stages its delta; a DEK-holding coordinator or owner verifies confinement before finalizing the root. The sub-agent cannot unilaterally publish it.
3. **Shard transfer:** Relay Nodes cannot infer object paths from opaque shard IDs. They check token validity, access mode, and byte quota, but cannot prove a write's path containment from its shard ID. Out-of-prefix uploads remain bounded by quota and orphan garbage collection; they do not acquire manifest publication authority merely by reaching a relay.

Direct-client grants follow the separate whole-volume and owner-publication restrictions in Direct-Client Grants.

7 Volume Lifecycle

This section specifies the end-to-end lifecycle of a volume, distinguishing control-plane (on-chain) and data-plane (CBFS) steps.

7.1 Create (control plane)

The owner submits a `create_volume` transaction to the Storage Manager (`0x0A`) with `volume_name`, `erasure_k`, `erasure_m`, `visibility`, and an initial CBY reserve. The actor:

1. Derives `volume_id = keccak256(owner_address || utf8_bytes(volume_name))`. If the volume already exists, revert with `VolumeAlreadyExists`.
2. Charges `VOLUME_CREATION_FEE = 1,000 CBY` from the owner's account.
3. For `PRIVATE` volumes, the client MUST generate the DEK off-chain and wrap it to the current CBSS threshold-IBE committee key. The request MUST include non-zero `cbss_committee_epoch` and `cbss_release_key_material_hash`; the Storage Manager

MUST validate them against Secrets Manager (`0x04`) before recording the wrapped key. Plaintext DEK material never enters consensus.

4. Writes a `StorageCommitment` record with status `ACTIVE`, empty `manifest_root`, `effective_size_bytes = 0`.
5. Assigns initial Relay Nodes via the Relay Registry.

7.2 Attach (control plane)

When a job is dispatched with volume attachments, the Storage Manager mints a runner CapToken per attachment. For private volumes, Secrets Manager (`0x04`) coordinates release of the DEK sealed directly to the runner's ephemeral X25519 key. Dispatcher carries only sealed material and never sees plaintext. The runner decrypts the sealed DEK on job start and zeroes it on completion.

7.3 Write (object transfer)

An authorized runner or client:

1. Generates a fresh 16-byte `write_id` and computes `shard_id = BLAKE3(volume_id || object_path || write_id)`.
2. For a private volume, encrypts the object under the Encryption rules; public-volume objects remain plaintext.
3. Reed-Solomon encodes the stored object bytes (ciphertext for private volumes, plaintext for public volumes) to K+M shards, padding to a multiple of K.
4. Selects K+M Relay Nodes from the cached Relay Registry and issues `PutShard(shard_id, shard_index, shard_bytes, metadata)` RPCs in parallel, each authenticated by the CapToken.
5. Records the new `ObjectDescriptor` in the local manifest and constructs the `PlacementRecord`.

Every step is local or off-chain. No transaction is submitted.

7.4 Read (object transfer)

1. Fetch the manifest root from the authoritative Storage Manager state (or a cached, freshness-bounded value).
2. Fetch and verify the manifest DAG from its committed root, using the volume-bound, locator-derived shard addresses specified in Manifest; decrypt private-volume nodes with the DEK. Public readers MAY use the verified `GET_MANIFEST` alternative described there.

© Cowboy Labs, Inc. 2026

3. Resolve `object_path` to an `ObjectDescriptor`; fetch its `PlacementRecord` via `GetPlacement`.
4. Request any K of K+M shards in parallel (prefer low-latency, fail over on timeout or corruption).
5. Verify shard hashes, reconstruct the stored bytes via Reed-Solomon, and truncate padding to the recorded size. For private volumes, verify `ciphertext_hash` before decryption.
6. For private volumes, decrypt; for all volumes, verify the plaintext `content_hash`.

7.5 Commit and Publication

Manifest publication follows CIP-9 §12.2:

1. **Publish data:** store the manifest DAG's added nodes at their volume-bound, locator-derived shard IDs as specified in Manifest. Publish their PlacementRecords to every assigned Relay Node via `PutPlacement`; manifest-node placement publication MUST succeed on every assignee before step 2. Nodes reachable from the previous root MUST remain intact while that root is authoritative.
2. **Commit authority:** submit the new root, `prev_root`, and added/removed shard deltas through the authorized control-plane path. The chain MUST compare `prev_root` with the current root before advancing `StorageCommitment.manifest_root`; in the same state transition, shard deltas update the native RAS inventory and `live_shard_count`. Each added and removed shard reference carries its real nonzero `chunk_root`; a removal must match the existing inventory row's root and size. PoR snapshots the selected inventory row's chunk-root at challenge issuance. Owner control-plane writes use the write-relayer transport specified in CIP-9 §12.2.1. Scoped runner commits follow Scoping and Concurrency; direct-client grantees cannot publish a root.

The required order is to publish new nodes, then advance the on-chain root. If the chain commit fails, the previous root and its nodes remain usable. Reclamation MUST protect both the current committed shard set and newly published bytes awaiting commitment. Every accepted publication or republication, including identical bytes at an existing node address, MUST establish a bounded tentative hold. A commit event predating that publication MUST NOT delete its bytes or clear its hold. The hold ends when a commit incorporating that publication makes the bytes live, or its deadline expires. Local publication and deletion MUST coordinate atomically so an older deletion decision cannot remove a concurrent replacement. Publishers MUST keep their holds valid through commitment and re-establish publication after a hold lapses.

Even after expiry, relays MUST preserve shards in the chain's current native RAS inventory for a volume that has not entered `GARBAGE_COLLECTING`, including private manifest nodes; they need

© Cowboy Labs, Inc. 2026

not decrypt the DAG to establish liveness. A stale removal event alone cannot authorize deletion, and uncertain eligibility MUST defer it. Finalized `GARBAGE_COLLECTING` status authorizes the terminal volume purge even while native rows remain for bounded consensus cleanup, subject to protecting shards still referenced by another live volume. CIP-9 §12.2 specifies the chain-state signal and publication rules. Orphan GC may reclaim abandoned uploads only under those rules. The CBFS removal-event implementation still has a tracked gap: COW-4300, recorded in CIP-9 §17.3.

The authoritative manifest is the one matching the root last accepted by the chain (READ_COMMITTED). Readers MUST verify that match. Root integrity and availability are separate: a committed root cannot supply bytes when too few relays serve its shards.

7.6 Delete, Grace, Restore

The owner MAY soft-delete a volume via `delete_volume`. The StorageCommitment transitions to `DELETED` for `VOLUME_DELETE_GRACE_EPOCHS = 1` storage epoch (`STORAGE_EPOCH_BLOCKS = 86,400`, ~24 h), during which:

- All active CapTokens are revoked.
- New CapTokens cannot be issued.
- Storage rent continues to accrue.
- The owner MAY call `undelete_volume` to restore status to `ACTIVE`.

If the grace window expires without restoration, the volume transitions to `GARBAGE_COLLECTING`. Relay Nodes learn of the transition via heartbeat acknowledgments or direct RPCs, purge that terminal volume's shards asynchronously, and storage rent ceases. After acknowledged event history is pruned, an independent, bounded per-block native GC lane removes one primary row and its dense ordinal per item, with durable progress and fair rotation among terminal volumes. It retains the `StorageCommitment` while rows remain and removes the volume from the live index only after `live_shard_count` reaches zero. Native-row GC progress continues across blocks, independent of daily rent epochs, without blocking or repeating economic settlement. Both PoR sampling builders exclude `GARBAGE_COLLECTING` volumes before drawing, even while native inventory rows remain. The same terminal state is reached when per-epoch billing detects a balance shortfall (see Settlement).

The current event-prune pass still stops at 1,024 acknowledged events per volume per daily storage epoch. A large history can delay entry into native GC for many epochs, and more than 1,024 new events per day can grow the backlog indefinitely. A separate, durable, fair per-block event-prune queue is required, but it cannot by itself guarantee a bounded backlog: valid manifest commits can create event-prune work faster than the current sweep's shared 32-item-per-block

© Cowboy Labs, Inc. 2026

ceiling. Indefinite production operation also requires a deterministic bound on outstanding prune work with commit admission and backpressure, prune capacity reserved at stage ingress so accepted staged commits are not blocked at finalize solely by prune saturation, and automatic client retry of temporary admission refusals. A permanently unavailable event-time relay presents a separate gate: the chain still requires that relay's signed event ACK before pruning, even after eviction, assignment rotation, or terminal volume status. A safe path to discharge that lost-relay obligation is also required. The queue, admission, retry, and lost-relay recovery changes remain proposed; the native inventory and its GC lane do not implement them.

8 Relay Nodes

A Relay Node is a dedicated storage participant distinct from validators and runners. It persists erasure-coded shards, serves reads and writes to authorized clients, participates in Relay-to-Relay repair, answers Proof-of-Retrievability challenges, and reports capacity and health to the chain.

8.1 Registration and Stake

A Relay Node registers by submitting a `register_relay(capacity_bytes)` transaction to the Relay Registry (`0x0B`) with a stake of at least `MIN_RELAY_STAKE = 33,000 CBY` (governance-tunable) and publishing its Ed25519 `relay_identity_pubkey`. The registration produces a **RelayNodeProfile**:

Field	Type	Meaning
`address`	`[u8; 32]`	Relay Node address
`stake_amount`	`u256`	Self-bonded CBY
`capacity_bytes`	`u64`	Advertised capacity
`used_bytes`	`u64`	Consensus-derived committed shard storage
`status`	`RelayStatus`	`ACTIVE`, `DRAINING`, `DISABLED`, or `INACTIVE`
`updated_epoch`	`u64`	Last profile mutation and heartbeat freshness
`shards_held`	`u32`	Active shard count
`shards_lost`	`u32`	Historical losses (reputation)

Field	Type	Meaning
`region_hint`	`[u8; 4]`	Geographic hint for placement diversity

Relay Nodes MUST refresh `updated_epoch` via `heartbeat()`. Placement considers only `ACTIVE` relays whose heartbeat remains fresh under the governance-controlled auto-drain policy. A stale heartbeat makes a relay eligible for auto-drain; `DRAINING`, `DISABLED`, and `INACTIVE` relays receive no new assignments.

A Relay Node MAY unstake after a `RELAY_UNSTAKE_DELAY = 86,400 blocks` (~24 h) cooldown starting after full drain completion (see Drain and Peer-Push).

8.2 Repair

Relay Nodes run a recurring repair loop every `REPAIR_CHECK_INTERVAL_SECS = 300` seconds:

8.2.1 Repair Local Shards

For each PlacementRecord where this node is an assignee, verify the local shard against its expected `shard_hash`. On missing or corrupt:

1. Fetch K healthy shards from peers listed in the PlacementRecord.
2. Reconstruct the ciphertext via Reed-Solomon, truncating to `ciphertext_size` (this step is why `ciphertext_size` is duplicated into the PlacementRecord — without it, padding would corrupt the reconstruction).
3. Re-encode, extract the expected shard, verify its hash, persist locally.

Self-heal is safe to run concurrently on any number of nodes: it only writes to local storage, never to the PlacementRecord.

8.2.2 Replace Unavailable Relays

For each PlacementRecord, probe all assignees. If a node is unreachable past a timeout:

1. **Leader election.** Among the live assignees, the node with the **lowest** `shard_index` is the unique leader for this PlacementRecord. Ties cannot occur (indices are distinct).
2. The leader selects a replacement Relay Node from the peer list (excluding current assignees) using the `NodeSelector`.

3. The leader reconstructs the missing shard(s) from surviving shards and pushes them to the replacement via `PutShard` or the signed `PeerRelayPushShard` RPC.
4. The leader writes a new PlacementRecord with the updated assignment and `version + 1` via a CAS `PutPlacement(shard_id, record, expected_version)`. If CAS fails (because another node replaced a different dead node in the same record concurrently), the leader retries in the next cycle.
5. Updated PlacementRecord is replicated to all current assignees via `ReplicatePlacement`.

Repair requires at least K healthy shards. Under the default K=4, M=2 scheme, three permanent shard losses before repair completes prevent reconstruction. Orphan shards without a referencing manifest MUST be garbage-collected after `ORPHAN_SHARD_TTL_SECS = 86,400` seconds.

8.3 Drain and Peer-Push

A relay may voluntarily exit (or be commanded to exit by governance) by transitioning to `Draining` status. A drain worker iterates assigned shards and peer-pushes each via the signed RPCs:

- `PeerRelayPushShard(sender_node_id, timestamp, shard_id, shard_index, old_record_hash, shard_bytes_hash, shard_bytes, signature)`
- `PeerPlacementUpdate(sender_node_id, timestamp, old_record_hash, new_record_hash, new_record, signature)`

Both are signed by the sending relay's identity key over domain-tagged BLAKE3 digests (`cbfs:peer_push_shard:v1`, `cbfs:peer_placement_update:v1`) and reject messages whose timestamp is outside a 60 s window. On successful drain, the node writes a signed `RelaySelfDisableRequest` when `shards_remaining == 0` and exits the active set.

9 Proof of Retrievability

Because Relay Nodes are economically incentivized to silently discard shards (to claim rent without serving), RAS implements periodic **Proof of Retrievability (PoR)** challenges. Challenges are cheap to verify, expensive to fake, and cost Relay Nodes measurable stake when they fail.

9.1 Challenge Protocol

A challenge is triggered by an on-chain timer every `POR_CHALLENGE_INTERVAL = 7,200 blocks` (~2 h at 1 s/block). It selects a random `(shard_id, shard_index, byte_offset,`

`byte_length)` tuple targeting an active relay. Consensus point-reads the selected shard's live native RAS inventory row and records its `chunk_root`, `shard_size`, and publication event sequence in the challenge. The relay MUST respond within `POR_RESPONSE_WINDOW = 50 blocks` with:

- The requested byte range (ciphertext bytes) from the named shard.
- A `byte_range_proof` that folds the challenged ciphertext chunks up to the shard's `chunk_root` (`cbfs_erasure::chunk_root`).

The on-chain verifier:

1. Recomputes the chunk leaves from the submitted ciphertext bytes and verifies `byte_range_proof` resolves to a chunk-root.
2. Requires that chunk-root to equal the selected native inventory row's `snapshot_chunk_root` recorded at challenge issuance.

PoR anchors on the selected native inventory row's public per-shard chunk-root. Every verification input — `snapshot_chunk_root`, `shard_id`, `shard_index`, and the relay's ciphertext bytes — is public. Because data is encrypted before erasure coding and hashing, shard commitments are DEK-independent, so PoR for a private volume is verifiable on-chain without the DEK.

A relay that stored the shard answers from local disk. PoR proves the shard is *retrievable at challenge time*, backed by stake and repair — not that the relay stores it continuously: within the response window a relay could reconstruct the challenged range from its erasure peers, so PoR deters silent discard economically (a repeatedly missing relay can be evicted and slashed) rather than proving dedicated storage cryptographically. Other inventory updates during the response window do not change this challenge's `snapshot_chunk_root`; removal or replacement of the challenged shard voids an unanswered challenge, and a later re-addition cannot rewrite its snapshot. If the volume enters `GARBAGE_COLLECTING` after issue, the unanswered challenge also voids at settlement even while its native row remains during bounded cleanup. This refunds the full bond, releases the open slot, and causes no miss, fee, slash, or bounty. Reversible `DELETED` status does not void a challenge.

9.2 Outcomes

Outcome	Consequence
Valid response within window or congestion grace	Challenge passed; bond returned less `POR_CHALLENGE_FEE`; no slash

Outcome	Consequence
Invalid proof	Response transaction rejected; challenge remains open for a valid response or expiry settlement
Unanswered challenge after window and congestion grace	Miss alarm without stake slash; with slashing enabled the per-shard consecutive-miss count advances; bond returned less `POR_CHALLENGE_FEE`
Third consecutive miss with slashing enabled	Relay disabled, `shards_lost` increments once, and `RELAY_EVICTION_PENALTY` is slashed up to available stake; challenger gets full bond refund and may receive a bounded pool-funded bounty
Shard incarnation removed/replaced or volume enters `GARBAGE_COLLECTING`	Challenge voided; full bond refund and open-slot release, with no miss, fee, slash, or bounty

A governance-tunable 1% share of storage rent plus retained `POR_CHALLENGE_FEE` funds challenge issuance; a confirmed third consecutive miss may pay a challenger bounty from the funded pool. With slashing disabled, unanswered challenges emit an alarm without incrementing the consecutive-miss count or touching stake. The current settlement path does not charge `POR_MISS_PENALTY` or `POR_FRAUD_PENALTY` for a single miss or rejected invalid proof.

10 Billing, Rent, and Eviction

Storage rent is CBFS's primary economic mechanism: it creates a continuous cost for occupying Relay Node capacity, directly fundable by any account.

10.1 Fee Components

Fee	When Charged	Amount
Volume creation	`create_volume`	`VOLUME_CREATION_FEE = 1,000 CBY`
Attachment	`submit_task` with volume attachment	`BASE_ATTACHMENT_FEE = 100 CBY` per volume per job
Storage rent	Per epoch	Rounded-up MiB of effective size at the storage rate
Read transfer	Bounded funded-channel checkpoint	Newly acknowledged logical-shard MiB units × execution-time rate, within signed caps; genesis rate 66,700 nano-CBY/MiB. Writes are outside this read mechanism.

Storage rent and read-transfer charges are separate from the Cycles, Accesses, and Cells execution fee markets specified in the Technical Whitepaper §4 and §17; RAS does not introduce another on-chain meter. On-chain operations (creating StorageCommitments, writing manifest roots, posting revocations and opening/claiming/refunding read channels) consume Cycles, Accesses and Cells normally. The per-epoch storage rent is a separate ledger maintained by the Storage Manager, debited from a dedicated `balance_reserved` on the account; dedicated read-channel principal is a further separate ledger. Neither is a substitute for transaction gas.

10.2 Relay Earnings

Relay Nodes earn from two streams:

1. **Per-epoch storage rent.** The total per-epoch rent on a volume is split among the platform, PoR challenge pool, and relays:

```
billable_mib = ceil(effective_size / 2^20)
total_storage_fee = billable_mib
                    × STORAGE_FEE_PER_MIB_PER_EPOCH
relay_pool = total_storage_fee
             × STORAGE_FEE_RELAY_BPS / 10,000
age_i = min(shard_age_in_epochs_i,
```

```
                MAX_SHARD_AGE_FOR_WEIGHTING)
relay_weight_i = shard_count_i × age_i
```

The relay pool is distributed in proportion to `relay_weight_i` under CIP-31 §5.

2. **Payer-authorized read transfer fees.** A fixed serving relay claims newly acknowledged logical-shard MiB units from a dedicated funded channel, within cumulative payer caps. The execution-time rate is governance-controlled; `TRANSFER_FEE_PER_MIB = 66,700 nano-CBY` is the genesis rate (see Parameters), not a promise that small reads cover lifecycle gas. No unsolicited relay report debits a reader's account.

`STORAGE_FEE_PLATFORM_BPS = 10%` of gross storage rent is credited to the Platform Fee Account (`0x18`, CIP-31 §4); `STORAGE_FEE_CHALLENGE_POOL_BPS = 1%` funds PoR challenges; and `STORAGE_FEE_RELAY_BPS = 89%` flows to relays, weighted by shard count and shard age. Relays with high `shards_lost` values receive fewer assignments over time via the `NodeSelector` reputation score.

10.3 Settlement

At each epoch boundary, the Storage Manager runs `settle_volume_epoch()`:

1. Compute `epoch_fee = ceil(effective_size_bytes / 2^20) × STORAGE_FEE_PER_MIB_PER_EPOCH`.
2. If `balance_reserved` ≥ `epoch_fee`: charge, split 10% to the Platform Fee Account, 1% to the challenge pool, and 89% to relays weighted by shard count and shard age, then increment `last_billed_epoch`.
3. If `balance_reserved < epoch_fee`: mark the volume as entering `GRACE_PERIOD`, with reads permitted and writes denied.
4. If the volume has been in `GRACE_PERIOD` for `STORAGE_GRACE_EPOCHS = 1` epoch (`STORAGE_EPOCH_BLOCKS = 86,400 blocks`, ≈24 h at 1 s/block) without top-up, transition to `GARBAGE_COLLECTING` and broadcast a purge signal to holding relays.

Transfer principal uses payer-funded cumulative read channels, not unilateral Relay Node attestations or an ambient debit from a Runner/Actor account. A direct payer-signed opening reserves a fixed deposit for a fixed relay wallet/node and volume. Each complete logical shard contributes `ceil(bytes / 2^20)` units only after the payer verifies the independently trusted manifest length/hash and durably acknowledges it. The signed request to serve bytes is not spending authority, and payment never replaces the independent data capability.

A relay/epoch scheduling group is emitted in atomic pages of 1..32 sorted unique channels, with one latest cumulative payer ACK per selected channel. At execution, a claim pays `(cumulative_units - settled_units) × current_TRANSFER_FEE_PER_MIB`, bounded by the

fixed maximum rate, the remaining signed cumulative fee ceiling, and dedicated escrow. The fixed relay receives 100% of that principal; the transaction submitter cannot redirect it. Sequence/counters are consumed once even at zero price. One invalid entry rejects the entire page. Changed registry ownership, volume status or expired new-read eligibility cannot revoke an existing funded claim before its claim deadline.

The channel's separate read-stop and claim-deadline heights are fixed by its terms. Strictly after the claim deadline, a gas-paying submitter may return the remaining principal only to the fixed original payer. The channel/tombstone is permanent, preventing reuse of its network/payer/funding-nonce identity. `escrow_remaining + paid + refunded = original_deposit` must always hold. Rent escrow, storage reward pools and challenge funds do not fund these reads.

Opening, checkpoint and refund gas is additional: Cycle, Cell and Access tariff components all debit the same native CBY balance. One-shot logical data requests, durable full-deposit/opening-gas reservations, bounded unsigned-delivery exposure, and immutable lifetime budgets prevent restart or retry from creating fresh authority. Exact signed transactions must be journaled before submission. Unknown outcomes permit only exact-byte resubmission; neither a new nonce nor a repacked page is a substitute for finality. A matching finalized archive and consumed committed nonce are required to release a pending transaction lane. Saved signed ACKs may be resent without data reread or a new signature; verified unsigned intents require explicit recovery. Unverified lost bytes cannot be acknowledged.

Small tails may be uneconomic even with batching. Lifecycle evaluation includes opening, multiple checkpoint partitions, charged failures, refunds, locked capital and delivery risk; explicit per-page/epoch/lifetime gas and loss budgets must precede signing. The CIP-31 §2.1 measurement discussion distinguishes actual execution measurements from production quotes and does not approve a default gas-share threshold or lock window.

This is a separately activated, default-off extension with independent capability `funded_payer_cumulative_read` and RAS kinds 28/29/30; legacy kind 27, its signatures and journals are not reinterpreted. Full wire, authorization and recovery requirements are in CIP-9 §10.4.1–§10.4.2. Implementation is not deployment/activation approval or historical pool migration. A rollback must preserve already-funded claim/refund paths.

The mounted SDK keeps one immutable, one-shot object/read selection. Its payment journal does not retain payloads, so cache loss or an incomplete response is not transparently repaired by a second paid read. There is no free fallback, automatic relay replacement, journal reset or guarantee of last-shard fair exchange. Trusted-node state/finality JSON is an observation, not a cryptographic state proof and not a relay's evidence of delivery.

© Cowboy Labs, Inc. 2026

10.4 Comparison with State Rent

The mechanism intentionally mirrors the state rent system specified in Technical Whitepaper §4.4 and §17.5 — grace period, then warning, then eviction — but differs in kind:

Property	State rent (on-chain state)	Storage rent (CBFS volumes)
Target resource	QMDB key/value state	Relay Node shard capacity
Billed unit	Cells (bytes of consensus state)	Bytes of `effective_size` (post-erasure)
Grace period	7 rent-epochs	1 storage epoch (~24 h)
Eviction	Storage pruned, code + root preserved	Shards purged, `StorageCommitment` retained until GC completes
Restoration	Anyone may restore with original data + back-rent	Owner may `undelete_volume` during grace; not after GC

11 On-Chain Control Plane

Two system actors anchor RAS on Cowboy: the **Storage Manager** at `0x0A` and the **Relay Registry** at `0x0B` (both registered in the system-actor table — Technical Whitepaper §9). Their state and instruction surfaces are detailed below.

11.1 Storage Manager (`0x0A`)

The Storage Manager is the on-chain authority for volume ownership, CapToken issuance, revocation, and billing. Its state comprises:

- `ras:volume:{volume_id}`

  `StorageCommitment` (owner, manifest root, size, erasure K/M, relays, status).

- `ras:owner-name:{owner_address}:{keccak256(volume_name)}`

  `volume_id` (name→id index).

- `ras:token:{token_id}`

  `CapToken` (runner tokens, stored for validation).

- `ras:relay-assignments:{volume_id}`

  List of assigned relay addresses.

- `ras:account-summary:{owner_address}`

  `AccountStorageSummary` (total volumes, effective bytes, reserved balance).

- `ras:volume-billing:{volume_id}`

  `VolumeBillingState` (last_billed_epoch, delete_requested_epoch, accrued_fees).

- `ras:challenge:{challenge_id}:{wallet_address}`

  `ChallengeRecord` (single-use nonce for high-impact ops, 30 s TTL).

- `ras:revocation:{wallet_address}:{cbfs_public_key}`

  `DelegationRevocation` (timestamp).

The `StorageCommitment` records volume identity, ownership, roots, erasure settings, billing, visibility, access grants, and key-release bindings. Its wire schema is defined by CIP-9 §11.1; the following is a field summary:

```
volume_id:            bytes32     // keccak256(owner || volume_name)
owner:                Address
volume_name:          String      // UTF-8, ≤64 bytes
visibility:           PRIVATE | PUBLIC
created_at:           u64         // block height
wrapped_dek:          bytes       // wrapped DEK; empty for PUBLIC
wrapping_key_hash:    bytes32
manifest_root:        bytes32     // manifest DAG root
live_shard_count:    u64         // dense native inventory row count
mount_allowed_actors: MountGrant[]
dek_version:          u64         // binds direct-client key packages
raw_size_bytes:       u64
effective_size_bytes: u64         // raw × (K+M)/K
erasure_k:            u8
erasure_m:            u8
last_updated:         u64         // block height of last manifest commit
degraded_shards:      u16
status:               ACTIVE | GRACE_PERIOD | DELETED | GARBAGE_COLLECTING
```

The authoritative live shard set resides in native RAS consensus inventory rather than this serialized commitment. A primary row keyed by `(volume_id, shard_id, shard_index)`

holds the shard's `chunk_root`, `shard_size`, publication event sequence, and dense ordinal; an ordinal row provides bounded selection and paging. `MAX_LIVE_SHARDS_PER_VOLUME = 1,048,576` bounds the live inventory, and one commit finalization may add and remove at most `MAX_COMMIT_FINALIZE_SHARDS = 65,536` shard references combined (CIP-9 §11.1).

The Storage Manager exposes `/ras/*` RPC endpoints on Cowboy validator nodes, including:

Endpoint	Purpose
`POST /ras/challenge`	Mint a single-use challenge (TTL 30 s) for high-impact mutations
`GET /ras/auth/me`	Verify delegation, return account summary
`POST /ras/auth/revoke`	Revoke a DelegationCert
`GET /ras/accounts/balance`	Check `balance_reserved` and current usage
`POST /ras/volumes`	Register a new volume (requires challenge)
`GET /ras/volumes`	List owned volumes
`GET /ras/volumes/{id}`	Volume details, relay endpoints, status
`GET /ras/volumes/{name}/open`	Open metadata; cross-owner clients specify `?owner={address}` and require a matching direct-client grant
`POST /ras/volumes/{id}/delete`	Soft-delete (enter grace)
`POST /ras/volumes/{id}/undelete`	Cancel soft-delete within grace

Owner control-plane mutations require the canonical owner authorization and write-relayer transport in CIP-9 §12.2.1. HTTP delegation envelopes use `X-Cbfs-Delegation`, `X-Cbfs-Timestamp`, `X-Cbfs-Nonce` or `X-Cbfs-Challenge-Id`, and `X-Cbfs-Signature`; where this envelope is used, timestamps MUST be within ±30 seconds and replay checks MUST reject reused nonces or challenges. A data-plane grant cannot substitute for owner control-plane authorization.

11.2 Relay Registry (0x0B)

The Relay Registry manages Relay Node membership, health, and settlement:

Key	Value
`ras:relay:{relay_address}`	`RelayNodeProfile` (by address)
`ras:relay-node:{node_id}`	`RelayNodeProfile` (by node_id)
`ras:active-relays`	List of active relay addresses
`ras:settlement:{relay_address}`	`RelaySettlementState` (claimable rewards)

Relay Nodes perform `heartbeat()`, `advertise_capacity()`, `report_shards()`, and `claim_settlement()` against this actor. It is also the target of PoR challenge resolution and slashing.

11.3 Cache Freshness

Relay Nodes maintain local caches of Storage Manager and Relay Registry state — ownership, delegations, grants, revocations, relay profiles — to avoid a chain round-trip on every CapToken validation. Caches refresh every **15 seconds** with a maximum staleness bound of **60 seconds**. Authorization MUST fail closed when required commitment, delegation, revocation, or grant state is stale or unavailable. A revoked DelegationCert is therefore effective across the network within 60 s of the on-chain posting. Relays missing from a local cache are fetched synchronously on first use, preventing a latency cliff for newly registered relays.

12 State Transition Function (Storage Subset)

This section specifies the storage-relevant portion of the protocol state transition function defined in the Technical Whitepaper (see *The State Transition Function*). Within each block:

1. **Process RAS system instructions.** Volume creation, deletion, undelete, commit manifest, revoke delegation, and settlement apply to Storage Manager state; relay registration, heartbeat, drain, and settlement apply to Relay Registry state.
2. **Issue runner CapTokens.** For each runner job assigned in the block with volume attachments, derive and persist the runner CapToken and sealed DEK copies.
3. **Resolve PoR challenges.** For challenges whose response window closes this block, verify submitted proofs and apply slashing and reputation updates.
4. **Epoch boundary (every `rent_epoch_length`).** Iterate all active volumes, compute storage rent, transition statuses (ACTIVE → GRACE_PERIOD → GARBAGE_COL-

LECTING, with restoration to ACTIVE permitted from GRACE_PERIOD), and distribute relay earnings.

Storage mutations pay execution fees under the Technical Whitepaper. Per-epoch rent is accounted for separately as described in Billing, Rent, and Eviction.

13 Security Considerations

13.1 Client-side encryption

For private volumes, Relay Nodes store ciphertext and observe structural metadata such as sizes and object counts. Decryption requires the DEK, delivered to authorized runners or direct clients under the applicable key-release contract. Public-volume objects and manifests are plaintext.

13.2 Shard ID opacity

`shard_id = BLAKE3(volume_id || object_path || write_id)` is a one-way hash. Relay Nodes cannot learn object paths, detect overwrites, or enumerate a volume from shard IDs alone. This enables privacy but forces Layer-3 path enforcement to be weak; see Scoping and Concurrency and Direct-Client Grants.

13.3 Durability and Scoped Writes

The Erasure Coding section defines the reconstruction threshold, and Repair defines recovery. The Scoping and Concurrency section separates shard upload authority from manifest publication authority. Private-volume prefix enforcement depends on the DEK-holding finalizer checking the staged delta before publication.

13.4 Relay eclipse and withholding

An object remains retrievable while at least K of its assigned shards are available; exceeding the erasure budget prevents reconstruction. PoR challenges detect silent withholding across the full relay set on a rolling basis. Readers detect a substituted or stale manifest by comparing its root with the on-chain `manifest_root`. Withholding instead prevents retrieval; a root commitment alone does not establish availability.

© Cowboy Labs, Inc. 2026

13.5 Sybil resistance

Relay registration requires a non-trivial `MIN_RELAY_STAKE`, and `NodeSelector` rewards diverse, healthy, reputationally clean relays. A sybil attacker must stake proportionally to the fraction of placements desired, and each sybil faces independent PoR slashing risk.

13.6 Key compromise

Compromise of the cold wallet key gives the attacker account authority, including the ability to authorize storage access. Confidentiality therefore depends on protecting both wallet authority and DEK material. Loss of the hot CBFS key requires revoking the DelegationCert on-chain (effective within the 60 s cache bound) and issuing a new one.

13.7 Key-delivery boundary

Runner key delivery follows Attach; cross-owner client key delivery follows Direct-Client Grants. Dispatcher MUST NOT receive a plaintext DEK. Revocation ends authorized access but cannot retract key material already delivered.

13.8 Replay and nonce reuse

DelegationCert signatures commit to `expires_at_ms`, `chain_id`, `network`, and `aud`; owner-token signatures commit to the version byte and full payload. Control-plane requests carry `X-Cbfs-Nonce` (LRU replay cache, 60 s TTL) or `X-Cbfs-Challenge-Id` (single-use, 30 s TTL). Peer-relay RPCs carry domain-tagged digests with a 60 s timestamp window.

14 Parameters (Genesis Defaults)

Storage pricing follows CIP-31. Billing and PoR economic values, minimum relay stake, and heartbeat/auto-drain thresholds are governance-tunable. Approximate block durations below assume one second per block.

Group	Parameter and default
Volumes	`MAX_VOLUMES_PER_ACCOUNT` = 256
Volumes	`MAX_VOLUME_SIZE` = 100 GiB

Group	Parameter and default
Volumes	`MAX_OBJECTS_PER_VOLUME` = 1,000,000
Volumes	`MAX_OBJECT_SIZE` = 1 GiB
Volumes	`MAX_VOLUME_NAME_LENGTH` = 64 bytes
Volumes	`MAX_OBJECT_PATH_LENGTH` = 512 bytes
Erasure coding	`DEFAULT_ERASURE_K` = 4
Erasure coding	`DEFAULT_ERASURE_M` = 2
Erasure coding	`MAX_ERASURE_K` = 16
Erasure coding	`MAX_ERASURE_M` = 8
CapTokens	`MIN_VOLUME_CAP_TOKEN_BLOCKS` = 600,000 (~7 days)
CapTokens	owner read/put TTL ≤ 900 s (`OWNER_TOKEN_OPEN_TTL_SECS`)
CapTokens	owner mount TTL ≤ 86,400 s / 24 h (`OWNER_TOKEN_MOUNT_TTL_SECS`)
CapTokens	owner-token clock skew ±30 s (`OWNER_TOKEN_CLOCK_SKEW_MS`)
CapTokens	DelegationCert expiry ≤ 30 days
CapTokens	`MAX_CAP_TOKEN_BYTES` = 16 KiB
Relay Nodes	`MIN_RELAY_STAKE` = 33,000 CBY (governance-tunable)
Relay Nodes	relay status = `ACTIVE`, `DRAINING`, `DISABLED`, `INACTIVE`
Relay Nodes	heartbeat freshness and auto-drain thresholds = governance-tunable
Relay Nodes	`RELAY_UNSTAKE_DELAY` = 86,400 blocks (~24 h)
Relay Nodes	`REPAIR_CHECK_INTERVAL_SECS` = 300 seconds
Relay Nodes	`ORPHAN_SHARD_TTL_SECS` = 86,400 seconds (~24 h)

© Cowboy Labs, Inc. 2026

Group	Parameter and default
Proof of Retrievability	`POR_CHALLENGE_INTERVAL` = 7,200 blocks (~2 h at 1 s/block)
Proof of Retrievability	`POR_RESPONSE_WINDOW` = 50 blocks (~50 s)
Proof of Retrievability	`POR_MISS_PENALTY` = 330 CBY
Proof of Retrievability	`POR_FRAUD_PENALTY` = 3,300 CBY
Proof of Retrievability	`RELAY_EVICTION_PENALTY` = 13,200 CBY
Proof of Retrievability	`RELAY_CHALLENGE_BOND` = 66 CBY
Proof of Retrievability	`POR_CHALLENGE_FEE` = 7 CBY
Proof of Retrievability	`CHALLENGER_BOUNTY` = 33 CBY
Proof of Retrievability	`MAX_SHARD_AGE_FOR_WEIGHTING` = 90 epochs
Billing	`VOLUME_CREATION_FEE` = 1,000 CBY
Billing	`BASE_ATTACHMENT_FEE` = 100 CBY
Billing	`STORAGE_FEE_PER_MIB_PER_EPOCH` = 2,905 nano-CBY
Billing	`TRANSFER_FEE_PER_MIB` = 66,700 nano-CBY
Billing	`MIN_STORAGE_BALANCE` = one epoch of fees at the current rate
Billing	fee split = 10% platform / 1% challenge pool / 89% relays
Billing	`STORAGE_GRACE_EPOCHS` = 1 storage epoch
Billing	`VOLUME_DELETE_GRACE_EPOCHS` = 1 storage epoch
Billing	`STORAGE_EPOCH_BLOCKS` = 86,400 blocks (~24 h)
Filesystem mount	`DEFAULT_SYNC_INTERVAL` = 5 s
Filesystem mount	`MIN_SYNC_INTERVAL` = 1 s
Filesystem mount	`MAX_LOCAL_CACHE_SIZE` = 10 GiB
Cache freshness (relays)	Refresh interval = 15 s

© Cowboy Labs, Inc. 2026

Cache freshness (relays)	max staleness = 60 s

15 Relationship to the Technical Whitepaper

This document layers on, but never supersedes, the Technical Whitepaper. Specifically:

- Actors, messages, timers, reentrancy, and per-handler resource limits follow Technical Whitepaper §3.
- Fee markets (Cycles, Accesses, Cells) and per-block basefee adjustment follow Technical Whitepaper §4 and §17. Storage operations that touch the chain consume these meters normally.
- State rent for **on-chain actor storage** follows Technical Whitepaper §4.4 and §17.5; storage rent in this document applies only to **off-chain CBFS volumes**.
- The Runner Marketplace (Technical Whitepaper §5) is the consumer of runner CapTokens. Job dispatch issues volume attachments through the Storage Manager.
- Consensus, randomness, and networking follow Technical Whitepaper §6. Relay Nodes use the consensus randomness beacon for PoR challenge targeting.
- Governance, upgrades, and system actor hot-code replacement follow Technical Whitepaper §11.
- Entitlements (Technical Whitepaper §15) MAY gate storage capabilities at the runner and actor manifest level (for example, whether an actor is permitted to open a volume at all); RAS CapTokens are the second, fine-grained layer within an entitlement's allowed scope.

Where this document and the Technical Whitepaper disagree on values or procedures, the Technical Whitepaper is the normative reference.

End of specification.

© Cowboy Labs, Inc. 2026